Phase 1 Static FJSP Mathematical Model

FJSP-AGV-RL Research Platform

June 1, 2026

1 Scope

This document describes the Phase 1 static flexible job shop scheduling problem (FJSP) model implemented by the repository. Transportation time, AGV routing, dynamic job arrivals, reinforcement learning, and graph neural network components are outside the Phase 1 scope.

2 Sets and Indices

Symbol	Meaning
J	set of jobs, indexed by j
O_j	ordered set of operations for job j, indexed by o
M	set of machines, indexed by m
A_{jo}	eligible machines for operation (j, o)

Operation (j, o) must precede $(j, o + 1)$ for every job j and operation position o where $o + 1 \in O_j$.

3 Parameters

Symbol	Meaning
p_{jom}	processing time of operation (j, o) on machine m
H	conservative scheduling horizon

The implementation uses a conservative horizon

$$H = \sum_{j \in J} \sum_{o \in O_j} \max_{m \in A_{jo}} p_{jom}.$$

This upper bound is not necessarily tight, but it is safe for the small CP-SAT baseline model.

4 Decision Variables

Here a and b denote operation identifiers such as (j, o). The explicit y variables are useful for a classical mixed-integer programming statement. The current OR-Tools implementation uses optional interval variables and `NoOverlap` instead of manually creating all y variables.

Symbol	Meaning
$x_{jom} \in \{0,1\}$	1 if operation (j,o) is assigned to machine m
$S_{jo} \geq 0$	start time of operation (j,o)
$C_{jo} \geq 0$	completion time of operation (j,o)
$C_{\max} \geq 0$	makespan
$y_{abm} \in \{0,1\}$	order variable for two operations a, b on machine m

5 Objective

The objective is to minimize the makespan:

$$\min C_{\max}.$$

6 Constraints

6.1 Machine Assignment

Each operation must be assigned to exactly one eligible machine:

$$\sum_{m \in A_{jo}} x_{jom} = 1, \qquad \forall j \in J,\ o \in O_j.$$

Machines outside A_{jo} are not valid choices and are never created as candidate assignments in the implementation.

6.2 Processing Time Link

If operation (j,o) is assigned to machine m, then its completion time must match its start time plus the selected processing time:

$$C_{jo} \geq S_{jo} + p_{jom} - H(1 - x_{jom}), \qquad \forall j, o,\ m \in A_{jo}.$$

6.3 Job Precedence

Operations within the same job must follow the given route order:

$$C_{jo} \leq S_{j,o+1}, \qquad \forall j \in J,\ o, o+1 \in O_j.$$

6.4 Machine Capacity

A machine can process at most one operation at a time. For any two distinct operations $a = (j,o)$ and $b = (k,q)$ that are both eligible for machine m, the disjunctive ordering is:

$$S_a + p_{am} \leq S_b + H(1 - y_{abm}) + H(2 - x_{am} - x_{bm}),$$

$$S_b + p_{bm} \leq S_a + H y_{abm} + H(2 - x_{am} - x_{bm}).$$

These constraints are active only when both operations are assigned to machine m. In the CP-SAT implementation, this logic is represented by optional intervals and a per-machine `NoOverlap` constraint.

6.5 Makespan Definition

The makespan must be no smaller than every final operation completion time:

$$C_{\max} \geq C_{j,last(j)}, \qquad \forall j \in J.$$

7 Feasibility Checker Mapping

The repository's feasibility checker validates the following model conditions:

- every operation appears exactly once;
- all job and operation ids are inside the instance range;
- every selected machine is eligible for the operation;
- recorded processing time matches the instance data;
- job precedence constraints are satisfied;
- no two operations overlap on the same machine;
- stored makespan equals the maximum recorded completion time.

8 Dispatching Rules

The strict dispatching rules use a common candidate tuple

$$(j, o, m, r_{jo}, s_{jom}, f_{jom}, p_{jom}),$$

where r_{jo} is the job ready time, s_{jom} is the earliest feasible start time on machine m, and $f_{jom} = s_{jom} + p_{jom}$ is the tentative finish time.

Rule	Operation selection	Machine selection
FIFO	smallest (r_{jo}, j, o)	earliest finish
SPT	smallest p_{jom} pair	selected with pair
EFT	smallest f_{jom} pair	selected with pair
MWKR	largest remaining route work	earliest finish
MOPNR	largest remaining operation count	earliest finish
Random	random ready pair	selected with pair

For MWKR, remaining route work is estimated as

$$W_{jo} = \sum_{q=o}^{last(j)} \min_{m \in A_{jq}} p_{jqm}.$$

For MOPNR, the ready job priority is its number of remaining operations:

$$N_{jo} = last(j) - o + 1.$$

9 Experiment Result Schema

Phase 1 experiment scripts write normalized CSV files under `experiments/results/`. Each row has the following fields:

The CSV layer is intentionally simple. It records what happened in each experiment without changing solver behavior or feasibility definitions. All Phase 1 experiment scripts use shared runner helpers that perform timing, call the solver, run the independent feasibility checker, collect failures, and copy optional solver metadata into the CSV record.

Field	Meaning
`experiment_id`	stable experiment identifier
`instance_name`	benchmark or toy instance name
`algorithm`	solver or dispatching rule name
`makespan`	objective value, empty if no schedule was returned
`feasible`	`PASS` or `FAIL` after feasibility checking
`runtime_s`	measured wall-clock runtime in seconds
`note`	solver status, error, or validation note
`objective_bound`	CP-SAT best objective bound, when available
`gap`	relative gap $(C_{\max} - LB)/\max(1, C_{\max})$, when available
`solver_wall_time`	CP-SAT internal wall time, when available

9.1 Batch Benchmark Entry Point

The batch experiment recursively discovers `*.fjs` files in stable path order. Each loaded static FJSP instance is evaluated with the selected Phase 1 heuristics and, by default, the CP-SAT reference solver. A configurable per-instance time limit bounds CP-SAT work on larger collections. Loader failures are retained as `LOAD` rows with `FAIL` status so that a malformed file cannot disappear silently from an experiment report.

9.2 Tracked FJSPLib Subset

The repository tracks a small real-data subset copied from the official ScheduleOpt FJSPLib repository under its CC-BY-SA-4.0 license: `https://scheduleopt.github.io/benchmarks/fjsplib`. The subset spans three source families and increasing instance sizes.

Instance	Jobs	Machines	FJSPLib BKS	Local CP-SAT
`fattahi1`	2	2	66	66
Hurink edata `ft06`	6	6	55	55
Brandimarte `mk01`	10	6	40	40
Brandimarte `mk03`	15	8	204	204

The local CP-SAT values above were independently validated by the feasibility checker and reported as `OPTIMAL`. The FJSPLib BKS column is upstream reference data, not a value inferred by this repository. For `mk03`, the local value matches the upstream BKS but CP-SAT reports `FEASIBLE` under the 10-second limit. The batch therefore records a time-limited feasible reference schedule, not a locally proven optimum.

10 CP-SAT Diagnostics

For CP-SAT results, the implementation stores three optional diagnostics in the common `ScheduleResult` object:

$$LB = \text{BestObjectiveBound}, \qquad gap = \frac{C_{\max} - LB}{\max(1, C_{\max})},$$

and the solver-reported wall time. For an optimal solution on the toy instance, the bound equals the makespan and the reported gap is zero. Heuristic results leave these fields empty.

10.1 CP-SAT Time-Limit Sweep

The tracked `mk03` sweep fixes the CP-SAT worker count to one and varies only the time limit. Fixing the worker count reduces parallel-search noise and makes the comparison easier to interpret.

Limit (s)	Makespan	Status	Bound	Gap
1	417	`FEASIBLE`	20	0.952038
3	285	`FEASIBLE`	20	0.929825
10	220	`FEASIBLE`	20	0.909091

Longer limits improve the feasible schedule in this run, but no row proves optimality. The default benchmark batch uses the solver's default worker configuration and is recorded separately. Worker count is part of the experimental configuration and must not be hidden during comparisons.

10.2 CP-SAT Worker-Count Sweep

The tracked `mk03` worker sweep fixes the time limit to 10 seconds and varies the CP-SAT worker configuration. A worker count of `auto` leaves the solver's worker choice at its default value.

Workers	Makespan	Status	Bound	Gap
1	220	`FEASIBLE`	20	0.909091
2	207	`FEASIBLE`	20	0.903382
4	207	`FEASIBLE`	20	0.903382
`auto`	209	`FEASIBLE`	75	0.641148

Every row passed the independent feasibility checker. The table is one timed observation, not a stable ranking of worker configurations: parallel CP-SAT search can vary between runs. In this observation, the automatic configuration produced the strongest bound, while two and four workers produced slightly better incumbents.

10.3 Repeated CP-SAT Worker Observations

The repeated experiment keeps the same `mk03` instance and 10-second limit, then runs each worker configuration three times. It stores every raw feasibility-checked run and a separate aggregate CSV. The reported standard deviation is the population standard deviation of the observed makespans.

Workers	Pass	Best	Mean	Std. dev.	Best bound	Best gap
1	3/3	220	220.000000	0.000000	20	0.909091
2	3/3	204	204.000000	0.000000	20	0.901961
4	3/3	204	204.000000	0.000000	21	0.897059
`auto`	3/3	204	204.000000	0.000000	88	0.568627

All 12 schedules passed the independent feasibility checker. Two, four, and automatic workers consistently reached the upstream BKS of 204 in this small sample. Because CP-SAT still reported `FEASIBLE`, these schedules remain time-limited feasible references rather than locally proven optima.

10.4 Repeated CP-SAT Benchmark Matrix

The controlled matrix recursively scans the tracked static FJSPLib subset. Every instance is evaluated twice with one worker and twice with the automatic worker configuration under a 3-second limit. Raw rows and per-instance aggregates are stored separately.

Instance	Workers	Pass	Best	Mean	Status	Bound
mk01	1	2/2	40	40.000000	OPTIMAL	40
mk01	auto	2/2	40	40.000000	OPTIMAL	40
mk03	1	2/2	286	286.000000	FEASIBLE	20
mk03	auto	2/2	236	238.500000	FEASIBLE	82
fattahi1	1/auto	4/4	66	66.000000	OPTIMAL	66
ft06	1/auto	4/4	55	55.000000	OPTIMAL	55

All 16 schedules passed the independent feasibility checker. The smaller tracked instances were solved optimally within the limit. On mk03, automatic workers improved the observed incumbents and bounds, but neither configuration proved optimality within 3 seconds.

10.5 Heuristic Benchmark Matrix

The heuristic matrix evaluates every deterministic Phase 1 rule once on each tracked FJSPLib instance. The two random families use fixed seeds 42, 123, and 999. Their ranges therefore describe reproducible observations rather than an uncontrolled stochastic sample.

Instance	Best deterministic rule	Random range	DispatchRandom range
mk01	DispatchMWKR (44)	148–165	92–125
mk03	DispatchFIFO/MWKR (204)	1086–1206	488–563
fattahi1	DispatchMWKR (66)	86–135	86–135
ft06	DispatchFIFO (55)	146–152	71–112

All 56 schedules passed the independent feasibility checker. No single rule dominates every instance: MWKR is strongest on mk01 and fattahi1, while FIFO is strongest on ft06 and ties MWKR on mk03. Ready-operation random dispatching is consistently better than the simple job-order random baseline in this tracked subset, but remains seed-sensitive.

10.6 BKS-Relative Heuristic Quality

The quality report reads structured upstream references from instances/fjsplib_bks.csv. For heuristic family h and instance i, it computes

$$gap_{\mathrm{BKS}}(h,i) = \frac{C_{\max}(h,i) - \mathrm{BKS}_i}{\mathrm{BKS}_i}.$$

The upstream BKS is a comparison reference, not a local optimality proof.

DispatchMWKR has the lowest mean best gap on the current four-instance subset. The sample remains deliberately small, so broader benchmark coverage should precede additional dispatching rules.

Algorithm family	Mean best gap	Maximum best gap
FIFO	2.780838	5.196078
SPT	1.601660	2.289216
EarliestFinishTime	0.475434	0.563636
DispatchFIFO	0.113258	0.303030
DispatchSPT	0.880437	1.465686
DispatchEFT	0.480214	0.578431
DispatchMWKR	0.047727	0.100000
DispatchMOPNR	0.159748	0.303030
Random	2.245276	4.323529
DispatchRandom	0.821524	1.392157

10.7 Tracked Benchmark Metadata Audit

Before expanding the benchmark subset, the metadata audit recursively discovers tracked FJSPLib files, matches them one-to-one with `instances/fjsplib_bks.csv`, and loads every discovered file. A row passes only if the tracked file exists, its upstream reference row exists, and the loader accepts the instance. All four currently tracked rows pass. This gate prevents silently omitted files and stale metadata from entering quality comparisons.